Q3: Trees and Random Forests (75 Marks)

Overview

In this question, you will implement and analyze decision trees and random forests for a sentiment classification task. You will complete coding tasks to build a decision tree from scratch, use Scikit-Learn to perform hyperparameter tuning, and interpret the learned models.

Dataset: Bag-of-words for Amazon Product Reviews

We will use a text sentiment classification task where each example is a plain-text online review from amazon.com. Our goal is to predict whether the sentiment is positive or negative from the text. The data has been preprocessed into a bag-of-words representation using a fixed vocabulary of 7729 terms.

- **Feature vector** $x_n \in \mathbb{R}^F$: A binary vector of size $F = 7729$, indicating which vocabulary terms are present in the review.
- **Label** $y_n \in \{0, 1\}$: A binary label indicating negative ($y_n = 0$) or positive ($y_n = 1$) sentiment.

The starter code repository includes a training set (6346 documents), a validation set (792 documents), and a test set (793 documents).

Part 1: Decision Tree Implementation from Scratch (20 Marks)

In this problem, you'll complete a from-scratch implementation of a Decision Tree. The starter code for this assignment can be found in the repository provided.

- **Code Task A:** Implement `predict` for a `LeafNode`. See the `LeafNode` class within starter code file: `tree_utils.py`.
- **Code Task B:** Implement `predict` for an `InternalDecisionNode`. See the `InternalDecisionNode` class within starter code file: `tree_utils.py`.
- **Code Task C:** Implement function to select the best binary split. See the starter code file: `select_best_binary_split.py`.
- **Code Task D:** Implement function to train the tree. See the starter code file: `train_tree.py`.

Part 2: Decision Trees for Review Classification (25 Marks)

We'll now examine decision trees for our sentiment classification problem using Scikit-Learn.

2.1 Train a Simple Tree and Visualize (10 Marks)

- Train a `DecisionTreeClassifier` with `criterion='gini'`, `max_depth=3`, `min_samples_leaf=1`, `min_samples_split=2` and `random_state=101`.
- Show the ASCII-text representation of your trained tree by calling the helper function `pretty_print_sklearn`.
- Is there any internal node that has two child leaf nodes corresponding to the same sentiment class? Why would having two children predict the same class make sense?

2.2 Hyperparameter Tuning (15 Marks)

- Perform a grid search for the hyperparameters of `DecisionTreeClassifier` over the following settings. Use `sklearn.model_selection.GridSearchCV` with `scoring = 'balanced_accuracy'`, the provided validation split (`cv=my_splitter`), `return_train_score=True`, and `refit=False`.
  - `max_depth` in `[2, 8, 32, 128]`
  - `min_samples_leaf` in `[1, 3, 9]`
  - `random_state` in `[101]`
- What are the values of `max_depth` and `min_samples_leaf` for the best tree found by the grid search?

Part 3: Random Forests for Review Classification (30 Marks)

We'll now see if an ensemble of decision trees can improve performance.

3.1 Train a Simple Random Forest and Analyze Features (10 Marks)

- Train a `RandomForestClassifier` with `max_depth=3` and `n_estimators=100` (and other hyperparameters from the starter code).
- Access the `feature_importances_` attribute of your trained forest.
- Create a two-panel table. The left panel should list the top 10 vocabulary words with the highest feature importance. The right panel should list 10 randomly chosen terms with near-zero importance (importance ¡ 0.00001), and state how many such terms were eligible.

3.2 Hyperparameter Tuning for Random Forest (10 Marks)

- Perform a grid search for `RandomForestClassifier` over the following settings:
  - `max_features` in `[3, 10, 33, 100, 333]`
  - `max_depth` in `[16, 32]`

  - `min_samples_leaf` in `[1]`
  - `n_estimators` in `[100]`
  - `random_state` in `[101]`

- What is the value of `max_features` of your best forest?
- What is the maximum possible value for `max_features` for this dataset? Why is it beneficial to tune this hyperparameter?
- When fitting random forests, what is the primary tradeoff controlled by the `n_estimators` hyperparameter? Can you overfit by setting it to be too large? Why or why not?

3.3 Final Model Comparison (10 Marks)

- Create a table summarizing the balanced accuracy for all four models developed (Simple DT, Best DT, Simple RF, Best RF). The table should have columns for model type, number of trees, max depth, and balanced accuracy on the train, validation, and test sets.
- Summarize your conclusions.

Good luck with the assignment